Robotics and Computer–Integrated Manufacturing 85 (2024) 102628

Contents lists available at ScienceDirect

Robotics and Computer-Integrated Manufacturing

journal homepage: www.elsevier.com/locate/rcim

# Deep reinforcement learning for solving resource constrained project scheduling problems with resource disruptions

Hongxia Cai, Yunqi Bian, Lilan Liu *

*Shanghai Key Laboratory of Intelligent Manufacturing and Robotics, School of Mechatronic Engineering and Automation, Shanghai University, Shanghai 200444, China*

ARTICLE INFO

*Keywords:*
Graph neural network
Reinforcement learning
Resource constrained project scheduling problem with resource disruptions

ABSTRACT

The resource-constrained project scheduling problem (RCPSP) is encountered in many fields, including manufacturing, supply chain, and construction. Nowadays, with the rapidly changing external environment and the emergence of new models such as smart manufacturing, it is more and more necessary to study RCPSP considering resource disruptions. A framework based on reinforcement learning (RL) and graph neural network (GNN) is proposed to solve RCPSP and further solve the RCPSP with resource disruptions (RCPSP-RD) on this basis. The scheduling process is formulated as sequential decision-making problems. Based on that, Markov decision process (MDP) models are developed for RL to learn scheduling policies. A GNN-based structure is proposed to extract features from problems and map them to action probability distributions by policy network. To optimize the scheduling policy, proximal policy optimization (PPO) is applied to train the model end-to-end. Computational results on benchmark instances show that the RL-GNN algorithm achieves competitive performance compared with some widely used methods.

## 1. Introduction

The resource-constrained project scheduling problem (RCPSP) [1] aims to minimize the duration of a project, subject to satisfying precedence constraints and resource availability constraints. Other scheduling problems, such as job-shop, flow-shop, and open-shop scheduling problems, can be viewed as special cases of RCPSP [2], and they have applications in many areas such as manufacturing, software development, construction, production scheduling, R&D, supply chain and services [3–5].

As an NP-hard problem [6], RCPSP has been widely discussed since the late 1960s [7]. In the last few decades, precise algorithms [8,9] and heuristic methods [10,11] have been proposed to solve RCPSP. In addition, extended definitions of the basic RCPSP have been proposed to solve a wide range of real-world problems, such as multiple modes for the activities, generalized precedence constraints, and alternative objectives [12–15].

In the context of Industry 4.0, by integrating technologies such as Cyber-Physical Systems (CPS), the Internet of Things (IoT) and the Internet of Services (IoS) into realistic production environments and manufacturing models, scheduling needs to take into account many dynamic events and the impact they have on scheduling [16,17]. However, many studies have hypothesized that resources and activity duration in RCPSP are deterministic, which is almost impossible in the real world. Projects often do not proceed as planned due to uncontrollable factors such as work errors and resources insufficient. This often results in calculating scheduling solutions according to deterministic models that may not be effective. A common uncertainty factor is resource availability [18]. For example, material delays, robots breakdown, and workers unavailable may cause jobs/activities delays, which makes it necessary to reschedule the interrupted jobs/activities.

In recent years, various reinforcement learning (RL) methods have been used to solve scheduling problems such as job shop scheduling problem (JSSP) and flexible job shop scheduling problem (FJSSP), and have achieved competitive results compared to traditional algorithms [19,20]. While traditional algorithms need to manually design heuristic algorithms for different problems, RL could automatically learn scheduling strategy. Some research has incorporated GNN into RL to solve graph-structure problems, in which GNN is used to extract features from graph-structured data. Compared to other feature extraction methods such as CNN [21] and MLP [22], GNN has the advantage of applying to graphs of different sizes without changing the network structure, and research has shown that GNN structure has good generalization ability in JSSP [23]. Although RCPSP is a widely encountered scheduling

* Corresponding author.
*E-mail address:* Lancy@shu.edu.cn (L. Liu).

https://doi.org/10.1016/j.rcim.2023.102628
Received 28 June 2022; Received in revised form 22 June 2023; Accepted 7 August 2023
Available online 21 August 2023
0736-5845/© 2023 Elsevier Ltd. All rights reserved.

problem, we have not found any literature which incorporated GNN into RL to solve RCPSP-related problems. That's the inspiration behind this research.

Our research approach is as follows: We extended the RL and GNN method to RCPSP and, after adjusting the parameters and evaluating the model performance, further extended this method to RCPSP-RD. We model RCPSP as sequential decision problems and formulate MDP for them. A GNN-based structure is implemented to extract useful features from different stages of the problem and map them to priority scores by policy network, to make scheduling decisions. This process is repeated in each stage of scheduling until the project is finished, then schedule timetable is generated based on history decisions. To optimize the scheduling policy, we use proximal policy optimization (PPO), an RL method that has been shown to have stable performance in many problems, to train the model end-to-end. We studied the impact of parameters on the learning performance of the model, and trained and tested our model using instances in the project scheduling problem library (PSPLIB). Based on our work on RCPSP, we further extended this method to RCPSP-RD. The original MDP was modified to take into account resource changes and activity interruptions in order to solve RCPSP-RD. We used the parameter settings that achieved good results in RCPSP, then trained and tested the model on the corresponding dataset. Experiments show that our algorithm was not only able to learn the generic scheduling policy of RCPSP, but also able to deal with RCPSP-RD, only by changing the training environment. Our algorithm also has competitive results in terms of computational time and makespan on both problems.

The main contributions of this paper are listed as follows:

1. A framework incorporating RL with GNN is proposed to solve the RCPSP, and then further solved the RCPSP-RD. The scheduling process is transformed to sequential decision-making problems and MDP is formulated for RL to learn scheduling policies. Then, A GNN-based structure was proposed to extract features from problems and map them to action probability distributions by policy network.
2. The model is trained on PSPLIB to learn scheduling policies. Experiments show that our model outperforms multiple algorithms and has good generalization performance.

The rest of the article is organized as follows. Section 2 provides a review of the literature. Section 3 describes the RCPSP and RCPSP-RD, then formulates MDP model for them. Section 4 describes the framework of our RL-GNN algorithm. Section 5 shows the parameters tuning process, training procedure and evaluation of our algorithm on benchmark instances. Section 6 discusses the limitations and applicable scenarios of our algorithm. Section 7 presents our conclusions and outlines our future research directions.

## 2. Literature review

### 2.1. Reinforcement learning for scheduling problems

Reinforcement learning has achieved good results on scheduling problems in recent years. A popular method is combining RL with some traditional algorithms, such as heuristics, variable neighborhood search, and Tabu search. Choi et al. [24] first introduced RL methods to the SRCPSP with dynamic project arrivals. The scheduling process was formulated as a MDP model, and they used Q-learning to remove the unimportant transition rules which were difficult to incorporate into heuristics. However, their method was not tested by traditional RCPSP nor with the scheduling problem with more activity. Jędrzejowicz and Ratajczak-Ropel [25] proposed a team of asynchronous agents for solving deterministic RCPSP with four heuristic algorithms and using RL to control the scheduling policy at different periods. They subsequently used a similar algorithm to solve the multi-modal RCPSP problem [26]. Sallam et al. [27] proposed an RL-based metaheuristic switching approach that combines multiple metaheuristic algorithms for solving SRCPSP. Shahrabi et al. [28] used RL with a Q-factor algorithm to improve the performance of the scheduling method based on variable neighborhood search, for the JSSP with random tasks and random machine faults. Li et al. [29] proposed a hybrid deep Q network to learn to choose the appropriate priority dispatching rules(PDRs) for solving FJSSP with insufficient transportation resources. They also considered new job insertions and machine breakdowns that could occur. In recent years, the research of multi-agent RL methods to solve scheduling problems, especially job shop scheduling related problems has gradually increased. Wang et al. [30] proposed a multi-agent RL method to solve JSSP in a resource preemption environment. Their main contribution was to build a multi-agent architecture and gave all agents a higher global reward to motivate them to learn to cooperate. Zhang et al. [31] constructed manufacturing equipment in the workshop as agents and proposed a self-organizing negotiation mechanism based on a contract network, for solving JSP in discrete manufacturing. Liu et al. [32] investigated a dynamic reentrant hybrid flow shop scheduling problem considering worker fatigue and skill levels. They constructed a self-organizing multi-agent system, designed a new reward function, and proposed an attention-based neural network to learn scheduling policy.

Another approach to solving such problems is the combination of RL with GNN. GNN is a type of neural network that can capture information from nodes and edges to extract features for the use of subsequent tasks. Because the information in scheduling problems can be represented by graphs, several researchers have extended the application of GNN to these problems. Park et al. [20] proposed a framework based on GNN and RL to solve the JSSP problem, incorporating a new GNN-based feature representation and feature learning method. They compared the results with various PDRs and investigated the computation time for handling problems of different sizes. Mao et al. [33] proposed a framework called Decima to schedule data processing jobs on distributed computing clusters. The framework extracted problem features and used an RL method based on a specially structured GNN to learn a scheduling policy, which is difficult to customize manually. Although both of these papers achieved good experimental results, their principles and reliability of GNN were not explained in detail. Zhang et al. [23] proposed a GNN-based method to embed information from a JSSP's disjunctive graph into the feature vector. They proposed an end-to-end framework based on GNN and RL to automatically learn the PDRs of a JSSP. On the basis of this research, Yuan et al. [34] designed a new state representation for JSSP and used invalid action masking technique to improve the model performance. Hameed and Schwung [21] cast JSSP as a distributed optimization problem and proposed a distributed learning scheme based on RL and GNN for it. They tested their algorithm on two manufacturing scenarios and discussed the applicability and performance of their method. Hu et al. [35] constructed an architecture consisting of a shared graph neural network and policy network for solving multiple traveling salesman problem. They used a two-stage approach consisting of RL and a regular optimization method to train the model, and improved the traditional training method to reduce the variance of the gradient. Their approach outperforms integer linear programming and heuristic algorithms especially on large scale problems.

### 2.2. Works on stochastic resource constrained project scheduling problem

In practice, for many reasons, some of the project variables may be stochastic, leading to stochastic RCPSP(SRCPSP). It is generally believed that in SRCPSP, the duration of activities is uncertain. The RCPSP-RD studied in this paper can be regarded as a special case of SRCPSP. Approaches to SRCPSP can be divided into robust scheduling (proactive and/or reactive) and stochastic scheduling. For robust scheduling, the uncertain information is usually taken into account in advance by using some means such as inserting time buffers into the schedule to make the

scheduling robust to any expected disturbances. Nevertheless, as the future is unpredictable, scheduling may still become unfeasible in some exceptional cases, so therefore a reactive scheduling is required. The reactive schedule method modifies the schedule to make it feasible in light of currently available information and minimizes deviations from the original baseline schedule [36–38]. Recent research have combined proactive and reactive procedures in an integrated framework [39–41].

Rather than obtaining a baseline schedule prior to project execution, as is done in robust scheduling, the stochastic scheduling approach sequentially builds a schedule using a specific scheduling policy [42]. A scheduling policy is a decision rule that determines which activity/activities to start at each decision point of the planning horizon, which can then be further classified as an open-loop or closed-loop policy [43]. Open-loop policies compute the scheduling plan for the entire project at once, while closed-loop policies make decisions only for the current situation.

The simplest and most popular method in an open-loop policy consists of various priority rules (PR), which play an important role in RCPSP extension problems. PRs are simple, intuitive, fast, and often used as part of more advanced heuristics, which require fewer computational resources. However, PRs are suffered so-called Graham's anomalies [44], and sometimes they do not achieve good scheduling solutions [1]. Except for PRs, many other methods were proposed these years, such as stochastic branch-and-bound [45], greedy heuristics [46], metaheuristics [47], tabu search [48], genetic algorithm [49,50], and a combination of multiple heuristics [51].

A closed-loop policy is more dynamic and adaptive than an open-loop policy, and can utilize the available information between each decision point to dynamically optimize the scheduling scheme during project execution [52]. A closed-loop policy is often based on a framework such as stochastic dynamic programming [53] and MDP [54,55]. However, the policy performance is limited due to the curse-of-dimensionality via the classical exact Bellman recursive solution approach, particularly in large instances. There are only a handful of papers using closed-loop policies. Choi et al. [56] developed a new way to combine heuristic solutions through dynamic programming in the state space that the heuristics generate, to solve SRCPSP with the uncertainties in task durations and costs, as well as uncertainties in the result of tasks. They then showed computational results for a 17-task example. Li and Womer [43] proposed an efficient ADP algorithm based on a roll-out strategy with constraint planning (CP) to improve performance. Xie et al. [57] proposed an ADP algorithm based on several heuristic rules to solve SRCPSP, the algorithm outperforms the priority-rule heuristics for test instances up to 120 activities.

Although RL has achieved good results in many scheduling problems, there are few studies about solving dynamic RCPSP with it. To our knowledge, there is no literature that uses a combination of GNN and RL to solve RCPSP-RD. Inspired by some methods in related research for solving scheduling problems [33,34], we propose a framework for RCPSP-RD, and show that the combination of GNN and RL can also achieve good results on it and has its own advantages.

## 3. MDP formulation

### 3.1. Problem description and assumptions

Consider a project described by an AON network $G = (V,E)$, where $V = \{0, 1, ..., n+1\}$ is a set of activities, and the duration of each activity $j \in V$ is $d_j$. $D = (d_0, d_1, ..., d_{n+1})$ denotes the duration of all activities. $E$ is directed edges and denotes the set of precedence among activities, e.g., a pair $(j,k)$ means that activity $j$ can be started only after activity $k$ has been finished. Furthermore, activity $j$ can only be started when all of its predecessors have been finished. $R_j$ denotes the requirement of resource capacity to start an activity$j$. There are $K$ renewable resources required for project execution and the requirement of activity $j$ for resource $k \in \{1, 2, ..., K\}$ is $r_{jk}$, and therefore $R_j = (r_{j1}, r_{j2}, ..., r_{jK})$. Assume that all resources are renewable, that is, the resources that each activity occupies are reusable once they are finished. $R^{free}$ denotes the amount of resources that have not been allocated to any activity.

In RCPSP the resource capacity $R_k$ of resource $k \in \{1, 2, ..., K\}$ is deterministic. For RCPSP with resource disruptions, it is assumed that resource capacity will change over time, and the probability distribution of each resource is known. The capacity of the resource $k \in \{1, 2, ..., K\}$ at period $t \in \{0, 1, 2..., T\}$ is $R_{kt}$, where T is the last period. To ensure that changes in resource capacity do not lead to unfeasible problems, assume that the minimum capacity of the resource $k$ is $LR_k = \max_{j \in V}\{r_{jk}\}$. Let activity 0 and $n+1$ represent the dummy start and end of the project, respectively, and their execution does not consume either time or resources. The project is finished only when all activities have been finished.

For RCPSP with resource disruptions, reduction of resources may result in the temporary suspension of some ongoing activities. Suppose that only resource capacity is insufficient to cause an activity to be interrupted, otherwise the activity cannot be interrupted once it has started. Assume preempt-resume rule hold, that is, an interrupted activity only needs to be executed for the remaining time after resuming, and no additional time is required. Assume activities that have been interrupted do not have a higher priority than other activities and can be resumed at any time once resources are sufficient. The scheduling objective is to minimize makespan.

In summary, RCPSP is based on the following assumptions:

1. Each activity only has one execution mode.
2. All resources are renewable.
3. Before an activity can be started, all its predecessors must be finished.
4. An activity can be started immediately when its predecessors have been finished.

For RCPSP-RD, additional assumptions are as follows:

1. Resource capacity will change over time and its probability distribution is known.
2. Once an activity has been started, it cannot be interrupted, except for insufficient resource capacity.
3. Interrupted activities can be resumed at any time when resources are sufficient and need to be executed for the remaining time.

### 3.2. MDP formulation

We formulated the scheduling process of RCPSP and RCPSP-RD as discrete-time MDP (DT-MDP). It is mainly based on the serial schedule generation scheme (SGS) described in [58], and is adjusted according to the possible interruption of activities for RCPSP-RD. For the two problems, except for the different transition processes, all other parts are the same. Normally, the time defined in the MDP is discrete. We define it as 1 unit between two adjacent time points. We use this model to iteratively compute the state of all activities and resources at each time point, from the initial state to the final state, and generate a feasible schedule scheme according to the actions taken by the scheduling policy in this process.

1. Stage

In most cases, a new stage is started when one or more activities can be scheduled.

An activity can be scheduled only when all of its predecessors have been finished and available resources are sufficient. The set of these activities at a certain time point called pending activities set, and is denoted by $\chi$. In the defined MDP, when $\chi$ is not empty at a certain time

point, the $i$th stage is started and the current time of this state is denoted by $t_i$, obviously $t_0 = 0$. If $\chi$ is empty, the status of all activities in the next time point will be computed and $\chi$ is checked, this processed is repeated until it is not empty, then a new stage is started. More information on stage-triggering conditions can be found in Section 4.2.1.

2. State

$S_i$ is the state of stage $i$. Denote the information used for the decision making, including the status of all activities, the status of resources and the set of pending activities, as

$$S_i = \{A_0, A_1, ..., A_{(n+1)}, R^{free}, R_{kt}, \chi\} \quad (1)$$

where $A_j = \{c_j, \bar{d}_j, R_j\}$. $A_j$ represents the status of the activity $j \in V$ at this time point, where $c_j$ is the status of the activity $j$. $c_j = 0$ denote that activity $j$ has not started or is interrupted, or has completed, and $c_j = 1$ if the activity is ongoing. $\bar{d}_j$ denotes the residual duration for the activity $j$ to finish. In the case of $\bar{d}_j = d_j$, activity $j$ have not yet started. And for $\bar{d}_j = 0$, activity $j$ is finished. $R_j = (r_{j1}, r_{j2}, ..., r_{jk})$ denotes the number of all resources required for activity $j$ to start. $R^{free}$ and $R_{kt}$ denote resources that have not been allocated and the resource capacity at this time point, respectively. $\chi$ is the set of all activities that can be scheduled. The initial stage's state $S_0$ is determined by the information of scheduling problem.

3. Action and policy

An action $x_i \in X_i$ denotes an activity selected from $\chi$ to be started in $i$th stage. For state $S_i$, a stochastic policy $\pi(x_i|S_i)$ outputs a probability distribution over the actions in $X_i$.

It is important to emphasize that an action is a single activity but not an activity set. If there are still sufficient $R^{free}$ to start more activities after an activity has been scheduled, the MDP model will transform to the next stage $S_{i+1}$ and will select an activity from $\chi$ as action $x_{i+1}$. Note that the current time of stage $i$ and $i+1$ are the same in this situation, i.e., $t_i = t_{i+1}$. This process will be repeated until $\chi$ is empty, means that it is default to allocate resources as much as possible at every time point.

4. Transition process

The transfer function $S^M(\cdot)$ compute the state of the next stage $i+1$ based on $S_i$ and the $x_i$:

$$S_{i+1} = S^M(S_i, x_i). \quad (2)$$

Assuming that the Markov property holds, means that the next stage $S_{i+1}$ depends only on the current state $S_i$ and the decision made $x_i$, independent of past states and decisions. For RCPSP, the MDP model will compute the state of the next stage based on information including currently ongoing nodes and remaining resources, etc. For RCPSP with the transition process, the model will consider the change in resources. If there are activities that have to be interpreted, the MDP model will stop all ongoing activities, release the occupied resource, and then start a new stage to reschedule activities. More details about this can be found in Section 4.2.2.

5. Reward

Our goal is to learn a scheduling policy that minimizes the makespan. The reward function is:

$$r(S_i, x_i, S_{i+1}) = -(t_{i+1} - t_i) \quad (3)$$

Designing an appropriate reward function for RCPSP is a difficult task, for we do not clearly know the effect of each action during scheduling a project. We use a simple method to compute the reward, maximizing the cumulative sum of rewards, therefor minimizing the makespan.

The entire sequential decision process of an MDP model for RCPSP and RCPSP-RD is depicted in Fig. 1. For the stagei's state $S_i$ with the current time of $t_i$, the scheduling policy provides an action $x_i$ according to a probability distribution over the actions in $X_i$. Then the state of the environment moves onto the next stage $S_{i+1}$. This process is repeated, from the initial state $S_0$ to the final state $S_N$. The objective is to find the optimal policy $\pi(x_i|S_i)$ to determine the minimum makespan.

## 4. Methodology

In this section, we describe how we adopted a neural network structure to implement the MDP in Section 3.2. We construct a scheduler (agent) using GNN and policy network to implement the policy function $\pi(x_i|S_i)$ in the MDP model. It can decide which activity to start by observing the state provided by the simulator (environment). The simulator will receive the action and compute the state of the next stage $S_{i+1}$ and return it to the scheduler. The two modules interact with each other to solve the problem, as shown in Fig. 2.

### 4.1. Construction of agent

In this section, we describe the process of obtaining the action $x_i$ based on the state $S_i$. There are two stages in this process. The node embedding $h_j^{(K)}$ is first extracted from state $S_i$ using a GNN, and then the policy network computes the probability distribution of all schedulable activities based on the node embedding and then makes the appropriate decisions.

#### 4.1.1. Extracting node embeddings using GNN

GNN [59] is a neural network that can learn graph-structured data representations. Node embedding is a method to represent nodes in continuous vector space, and it takes into account the characteristics of nodes and edges and retains different types of relationship information in the graph. The purpose of extracting node embedding is to represent nodes using low-dimensional vectors that contain information about the target nodes themselves and the topology of the network, which can be used to perform a variety of endpoint tasks [60,61]. Most of the GNN computes the embedding vectors in an iterative way, through the mechanism called message passing and aggregation. In each iteration, information is propagated through the pathway between nodes, and then all nodes aggregate the information by aggregation function.

To use the GNN, we consider each activity in the RCPSP as a node in a network. The constraints between activities are graphed as edges between nodes (point from parent nodes to child nodes), and the feature of node $j$ is $A_j$. Note that all information of the problem can be represented using this network except for $R^{free}$.

We use a Graph Isomorphism Network (GIN) [62] to compute the embedding vectors for each node in this network. The state information $A_j$ of each node is embedded in a set of vectors $h_j$, containing information of each node itself and nearby nodes for the subsequent decision-making process. Node information is propagated in the opposite direction of the precedence relationship between nodes, i.e., each node receives the information only from its child nodes and not from the parent nodes. The network is not transformed into an undirected acyclic graph because doing so would result in the loss of precedence information. Embedding

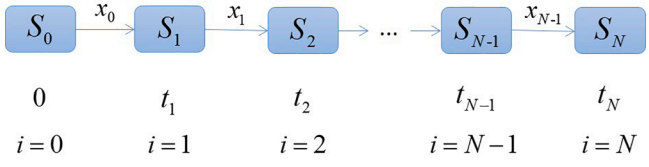



**Fig. 1.** Markov Decision Process for RCPSP and RCPSP-RD.

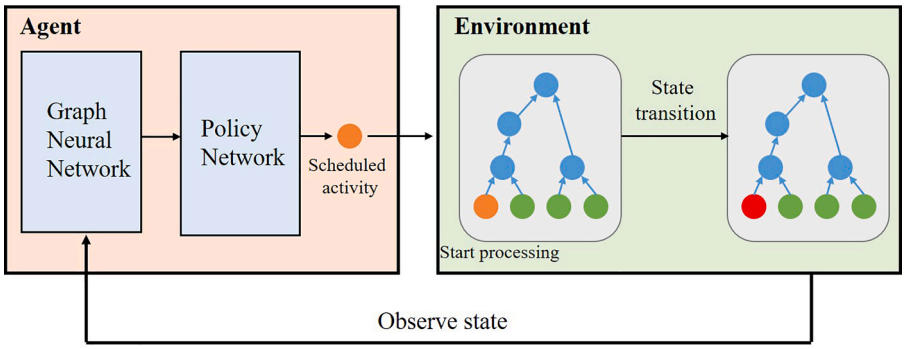



**Fig. 2.** Structure of agent and environment. Green nodes are activities that can be scheduled at this time point while blue nodes are not, red node is the done activity. Orange nodes denote processing activities. Agent observes state given by environment and selects an activity from$\chi$to be processed.

vectors are computed as

$$h_j^{(k)} = f_{\theta_k}^{(k)}\left[\left(1+\varepsilon^{(k)}\right)\cdot h_j^{(k-1)} + \sum_{u\in\xi(j)} h_u^{(k-1)}\right] \tag{4}$$

where $h_j^{(0)} = A_j$. $h_j^{(k)}$ is the $k$th level embedding of the node. $f_{\theta_k}^{(k)}$ is $k$th non-linear transformation of the input vector and is implemented by fully connected neural networks. $\varepsilon$ is an arbitrary number that can be learned. $\xi(j)$ represents all of the child nodes of node $j$.

We construct GNN by overlapping $K$ embedding layers. After the iterative update, the $K$th embedding vector $h_j^{(K)}$of each node is computed, as depicted in Fig. 3, where the solid arrows represent precedence constraints between nodes, and the dashed arrows point to the direction of information propagation. A global representation of the entire graph $h_G$ is needed for the subsequent state value estimation in the PPO algorithm. $h_G$ is computed by the pooling function $P$ taking the deepest embedding vector $h_j^{(K)}$as input. Here we use the average pooling function, as is shown in Eq. (5).

$$h_G = P\left(\left\{h_j^K : j\in V\right\}\right) = 1\Big/|V|\sum_{j\in V} h_j^K \tag{5}$$

There are two advantages to using GNN for feature extraction. First, this approach can be applied to scheduling problems with different sizes (number of activities). In contrast to some methods that create a feature vector containing all the state information according to specific rules [22,63], GNN method can map both the state and constraint information of the nodes in graphs into a set of equal-length embeddings, which greatly facilitates the processing of subsequent steps. Second, because the information stored in these embedding vectors is not hard-coded, we do not need to manually design the encoding method. In the subsequent end-to-end training, GNN can automatically learn how to extract useful information from the state and gradually improve performance.

#### 4.1.2. Decision-making with node embedding

The policy network take $h_j^{(K)}$ as input to compute the probability distribution of all actions and select an action $x_i$ based on it. An action contains only one node to be started, and if there are multiple nodes can be started, the process of generating an action will be repeated until no nodes can be scheduled. The current time of the environment will not be pushed forward until all resources have been allocated as much as possible. The purpose of this design is to minimize the action space thus reduce the difficulty of the subsequent training process.

The decision-making process is as follows. First, the embedding vector $h_l^K$ of all nodes in $\chi$ is concatenated with $R^{free}$ and $h_G$, then priority score is computed using Eq. (6)

$$s_l = q_{\theta_\pi}\left(\left[h_l^K \| R^{free} \| h_G\right]\right) \tag{6}$$

where $s_l$ is the priority score of node $l$ , $q_{\theta_\pi}$ is a no-linear function implemented by a neural network, $\|$ is the vector concatentation operator.

The probability distribution of all actions is computed using the softmax operation, as shown in Eq. (7)

$$P(node=l) = \frac{\exp(s_l)}{\sum_{u\in\chi}\exp(s_u)}. \tag{7}$$

A node is selected as action $x_i$ based on the action probability distribution in stage $i$. The policy network is used to map node features to priority scores, similar to the concept of calculating the longest path of a node in the critical path method and determining its priority. All the above operations are end-to-end differentiable.

### 4.2. Construction of the simulator

In this section, we describe the process of how the simulator receives the action $x_i$ and then compute the state of the next stage $S_{i+1}$. Starting

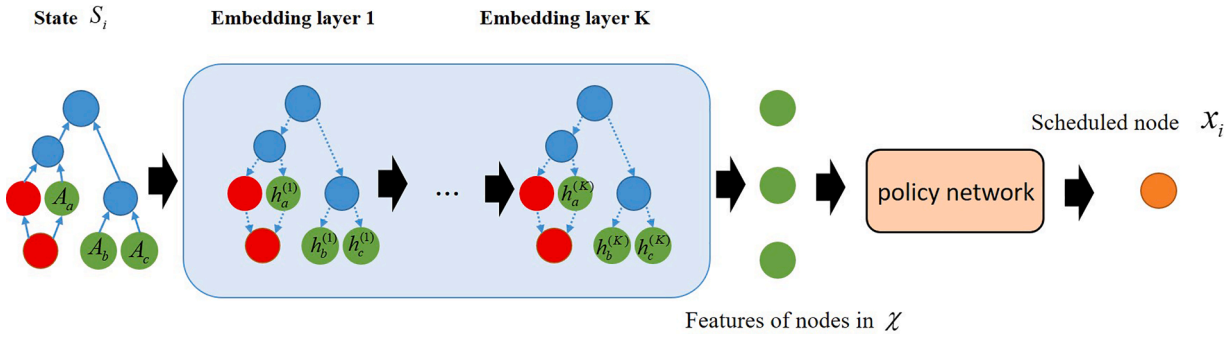



**Fig. 3.** Decision-making process. The original features of the nodes are input into the GNN to obtain the embedding $h^{(K)}$. Policy network takes $h_j^{(K)}, j\in\chi$ as input and output action $x_i$ for each stage.

**Algorithm 1**
Scheduling process for RCPSP-RD.

**Begin**
1: **While** project have not been finished **do**
2: **if** $\chi$ is case2 **then**
3: Add $\chi$ to ongoing nodes set $\rho$, $\rho = \rho \cup \chi$
4: Update $R^{free}$ and $\chi$
6: **end if**
7: **While** $\chi$ is case3 **do**
8: Receive action $x_i$ from scheduler
9: Add $x_i$ to $\rho$, $\rho = \rho \cup x_i$
10: Update $R^{free}$, $\chi$
11: **end for**
12: **While** $\chi$ is case1 **do**
13: Update the current time $t_{currenttime} = t_{currenttime} + 1$
14: Update $R^{free}$ according to $R_{kt}$
15: **if** $\exists r_k \in R_{free} < 0$ **do**
16: Interrupt all ongoing nodes$j, j \in \rho$
17: **Return** to line 1
18: **end if**
19: **for** each node$j, j \in \rho$ **do**
20: Update residual duration $\bar{d}_j = \bar{d}_j - 1$
21: **if** $\bar{d}_j = 0$ **do**
22: $c_j = 0$, return occupied resources
23: **end if**
24: Update $\chi$
25: **end for**
26: **end for**
27: **end for**
**End**

with the first stage, the simulator will iteratively compute the scheduling problem's state at each time point until a new stage is triggered, or until the problem is solved. This process mainly consists of two parts: triggering the stage and calculating the state of the next time point.

#### 4.2.1. Trigger conditions for stages

Each node must meet two conditions before it can be started:

1. Its parent nodes have finished.
2. All types of resources for this node are sufficient at the current time point.

The simulator computes $\chi$ at each time point based on these two conditions. Then, depending on $\chi$ and $R^{free}$, the following cases can be classified.

Case 1. $\chi$ is the empty set, i.e., no nodes can be started.

Case 2. All of the nodes in $\chi$ can be started.

Case 3. Not all of the nodes in $\chi$ can be started due to lack of $R^{free}$.

Only when the case 3 occur will a new stage be triggered, because we have to select which node to start. When case 2 occurs, all nodes in $\chi$ are started automatically but will not lead to a new stage.

#### 4.2.2. Compute the state of the next time point

After action $x_i$ has been taken, the simulator will iteratively compute the state information of nodes in each time point and check it until a new stage $S_{i+1}$ is triggered. In each cycle, the simulator updates the problem information, taking into account the states of all nodes, precedence relationships, and some other factors. The difference between RCPSP and RCPSP-RD is that the simulator of RCPSP-RD takes into account the impact of resource changes and activity interruptions. When the resources are insufficient, all ongoing nodes will be interrupted and added to $\chi$, and a new stage will be triggered to make a reschedule. This process is shown in Algorithm 1.

Compared to directly converting serial SGS into a Markov decision process, we have made an environment allocates as many resources to nodes as possible at each stage, and reduced some unnecessary actions of the agents. The purpose of designing such an environment is to reduce a large number of suboptimal actions that the agents do not need to explore, thereby reducing the total number of actions required to solve the problem. This allows the model to converge faster and achieve better results during training. However, the disadvantage of this approach is that it may cause the algorithm to miss optimal solutions on some problems, this phenomenon will be discussed in Section 5.

### 4.3. PPO algorithm

We use PPO [64], an actor-critic method in RL, to update the parameters in the scheduler. The actor refers to GNN and policy network, the critic shares the same GNN layer with actor, and uses a no-linear transformation $V_{\theta_\sigma}$, implemented by neural network, take $h_G$ as input to compute a scalar to estimate the cumulative rewards at $S_i$. We simulate the project scheduling process repeatedly and generate some independent trajectories, then update network parameters to improve the scheduling policy.

The PPO algorithm is a variant of the policy gradient method in RL and has greater stability than many other RL algorithms (which is one of the reasons we chose it). The basic strategy gradient method optimizes the strategy by updating all parameters $\Theta = \{\theta_1, \theta_2, ..., \theta_k, \theta_\pi, \theta_\sigma\}$ in the direction of the gradient of the objective function $\nabla L(\Theta)$ with

$$\Theta_{new} = \Theta_{old} + \eta \nabla_\Theta L(\Theta) \tag{8}$$

where $\eta$ is the learning rate,$L(\Theta) = \widehat{\mathbb{E}}_i[\sum_{x_i \in X_i} \log \pi_\Theta(x_i|S_i)\widehat{A}_i]$, $\widehat{A}_i$ is an estimator of the advantage function at timestep $i$.

Experiments has shown that updating the parameters using the original policy gradient method often leads to destructive large-scale policy updates [64], causing the parameters to deviate significantly from their original positions, thereby corrupting the previous training results. To overcome this drawback, researchers proposed trust region policy optimization (TRPO) [65], which uses a KL divergence constraint to limit the magnitude of parameter updates, but requires solving constrained optimization, which is computationally demanding. PPO simplifies the computation by using a clipped surrogate objective instead of a KL divergence constraint, and only updates the parameters when scheduling performance is improved. The clipped objective is formulated as

$$L_i^{CLIP}(\Theta) = \widehat{\mathbb{E}}_i[\min(R_i(\Theta)\widehat{A}_i, clip(r_i(\Theta), 1-\varepsilon, 1+\varepsilon)\widehat{A}_i)] \tag{9}$$

where $R_i(\Theta)$ denotes the probability ratio$R_i(\Theta) = \frac{\pi_\Theta(x_i|S_i)}{\pi_{\Theta_{old}}(x_i|S_i)}$. $\varepsilon$ is a hyperparameter. The function $clip(R_i(\Theta), 1-\varepsilon, 1+\varepsilon)$ restricts $R_i(\Theta)$ to $[1-\varepsilon, 1+\varepsilon]$. The generalized advantage estimation function $\widehat{A}_i$ is defined as

$$\widehat{A}_i = \delta_i + (\gamma\lambda)\delta_{i+1} + ... + (\gamma\lambda)^{T-i-1}\delta_{T-1} \tag{10}$$

where $\delta_i = r_i + \gamma V_\Theta(S_{i+1}) - V_\Theta(S_i)$ and $V_\Theta(S_i)$ is the value function to estimate state value. $T$ is the termination step of the episode.

The loss of the value network uses the mean squared error (MSE), which is formulated as

$$L_i^{VF}(\Theta) = \frac{1}{2}\left(V_\Theta(S_i) - V_i^{\text{targ}}\right)^2 \tag{11}$$

where $V_i^{\text{targ}} = \sum_{i=t}^{T} \gamma^{i-t} r_t$ is the cumulative reward.

Because the policy function and value function share some of the parameters, the loss function combines two errors: $L_i^{CLIP}$ and $L_i^{VF}$. We also add an entropy bonus to it in order to allow the agent to explore more efficiently, resulting in a combined loss function of

$$L_i^{CLIP+VF+S}(\Theta) = \widehat{\mathbb{E}}_i\left[L_i^{CLIP}(\Theta) - c_1 L_i^{VF}(\Theta) + c_2 S_i(\pi_\Theta)\right] \tag{12}$$

where $c_1$, $c_2$ are the coefficients, and $S_i(\pi_\Theta) = -\sum_{x_i \in X_i} \log(\pi_\Theta(S_i|x_i))$ $\pi_\Theta(S_i|x_i)$ is the entropy bonus to encourage the agent to explore more efficiently.

The algorithm updates the parameters $\Theta$ in the direction of $\nabla_\Theta L_i^{CLIP+VF+S}$ to maximize the cumulative reward, so that the agent learns to improve scheduling policy.

## 5. Experiment

We train and test the performance of our model using PSPLIB [66], which is a project scheduling problem library. PSPLIB was developed by Kolisch and Sprecher containing instances of different types and sizes, and has been widely used to test various exact and heuristic methods. There are four sizes of instances in the single-mode RCPSP set which were named j30, j60, j90 and j120, containing four different numbers of activities: 30, 60, 90, and 120, the corresponding numbers of their instances were 480, 480, 480, and 600. All the instances were generated by ProGen [67] with differing network complexity, resource factor, and resource intensity. In our experiment, our model was trained on a subset of instances from PSPLIB, with the remaining instances being used for validation and testing of the model's performance.

We designed experiments as follows: First, we performed an imitation learning experiment to test the performance of the model and determined its hyperparameters. We then trained and tested the model's performance on both RCPSP and RCPSP-RD. All the program was developed in Python with the neural networks being implemented using PyTorch. The hardware we used was two same machines with Intel(R) Xeon(R) W-1270P CPU and Nvidia Quadro RTX 5000 GPU with 16GB of RAM.

### 5.1. Imitation learning

Imitation learning techniques [68] aim to mimic human behavior in a given task. An agent is trained to perform a task from demonstrations by learning a mapping between observations and actions. In this experiment the training process was as follows: We collected optimal schedules for 90 instances in j30 and converted them into states, actions, and rewards corresponding to each stage in our model. These data was then used for supervised learning to train the model. We had three objectives for this experiment. 1. To test the model's ability to fit the data and its upper limit. 2. Based on this, to study the impact of hyperparameters on model performance and adjust them accordingly. 3. Subsequent reinforcement learning training will be carried out on the basis of pre-training the model with imitation learning, thereby accelerating the convergence speed of reinforcement learning.

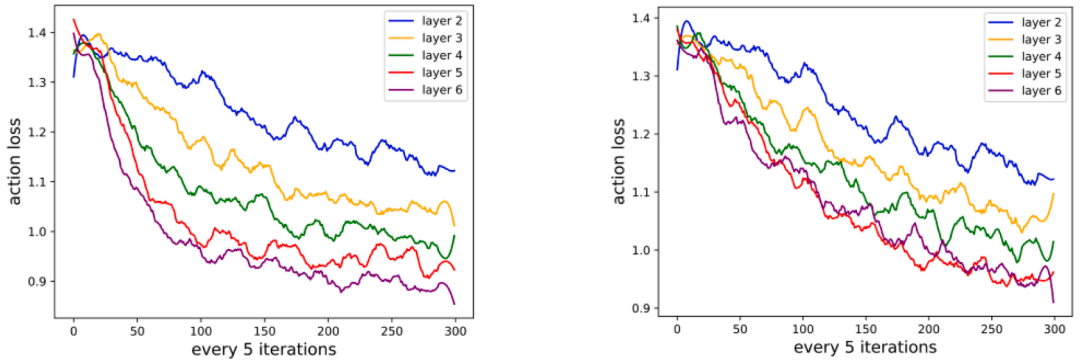
During the training process, we first set the network structure parameters, including the number of GNN layers and the dimensions of the actor network's hidden layer, as small as possible. For parameters related to reinforcement learning, we referred to the default settings in CleanRL [69]. We then increased one network structure parameter at a time and observed the loss curves of the actor and value networks during training. The experimental results show that changes to most parameters did not have a significant impact on the loss curves, including the hidden layers of the policy and value networks, the hidden dimensions of these layers, the hidden dimensions of the GNN layers, the pooling functions of the GNN, different types of the activation function. We also observed that setting different random seeds before training only affected the convergence speed of the action loss curve and have little impact on the final converged value. Only the number of GNN layers $K$ and the number of hidden layers in each GNN layer had a significant impact on model performance, as shown in Fig. 4(a) and (b).

As the number of GNN layers and GNN hidden layers increased, the action loss can be reduced through training, indicating that the model has stronger learning capabilities. However, when both exceed 5 layers, the improvement in model performance is no longer significant. Therefore, we performed a grid search between [2,5] for each parameter to explore the optimal combination of the two parameters. There was a total of 16 experiments, for ease of comparison, we only present the loss curves for the best few parameter combinations here. If the difference between the loss curves is small, the model with fewer total layers is chosen, as shown in Fig. 5(a) and (b).

Although increasing both the number of GNN layers and GNN hidden layers can reduce the final action loss, it can be seen from the value loss curve that as the complexity of the model increases, the value loss converges more slowly. In subsequent training on RCPSP instances, we found that because the value network guides the training of the actor network, the slowdown in the convergence speed of the value loss leads to a decline in model performance and even difficulty to convergence. Furthermore, we also found that when the number of GNN layers exceeds 5, further increasing the complexity of the model leads to a decline in model performance. Taking all factors into consideration, we chose to set both GNN layers and GNN hidden layers to 4. In summary, the parameters in our algorithm are shown in Table 1.

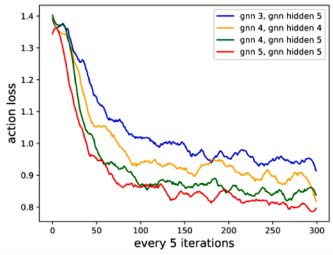
### 5.2. Training and testing on RCPSP

We trained and tested on RCPSP using the parameters described above. The experiment was arranged as follows: For each size of the instance, we used 120 instances of the corresponding size for training, 90 for validation, and 90 for testing model performance. We also took 60 instances from each size of the instance to form a mixed dataset for training. A total of 5 models were trained, corresponding to 5 datasets: j30, j60, j90, j120 dataset and mixed dataset. Algorithm 2 shows the

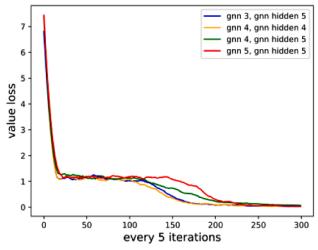



(a)action loss curves of increasing hidden layers in each GNN layer (b)action loss curves of increasing GNN layers

**Fig. 4.** Action loss curves of increasing GNN layer and increasing hidden layers in each GNN layer.




(a)action loss curve of parameter combination




(b)value loss curve of parameter combination

**Fig. 5.** Action loss and value loss of parameter combination.

**Table 1**
Setting of hyperparameters for PPO.

| Hyperparameter | Value | Hyperparameter | Value |
|---|---|---|---|
| Optimizer | Adam [70] | Number of episodes per update ($n$) | 10 |
| Parallel environments | 4 | Update epoch($k$) | 4 |
| Total timesteps | 300,000 | Layers of GNN($K$) | 4 |
| Learning rate ($\eta$) | $1 \times 10^{-5}$ | Hidden dimensions of GNN | 32 |
| Discount factor ($\gamma$) | 1.0 | Layer of policy network | 2 |
| GAE parameter ($\lambda$) | 0.95 | Hidden dimensions of policy network | 32 |
| Clipping parameter ($\varepsilon$) | 0.2 | Layer of value network | 3 |
| Value function coefficient ($c_1$) | 0.5 | Hidden dimensions of value network | 32 |
| Entropy bonus coefficient ($c_2$) | 0.01 | Collect steps of each environment | 32 |

**Algorithm 2**
PPO training process.

**Begin**
1: **for** training episode$1, 2, \ldots, m$ **do**
2: select $n$ instance from dataset
3: **for** instance$p = 1, 2, \ldots, n$ **do**
4: **for** episode $i = 1, 2, \ldots, T$ **do**
5: **if** the current state is non-trivial state **do**
5: compute action$x_i$, action probability distribution$\pi_\Theta(x_i|S_i)$
6: compute next state $S_{i+1}$ and reward $r_i$
7: collect transition sample$(S_i, x_i, r_i, S_{i+1})$
8: **end if**
9: **end for**
10: **end for**
11: **for** update epoch $1, 2, \ldots k$ **do**
12: Compute $L_{pi}^{VF}(\Theta), \ldots L_{pT}^{VF}(\Theta), L_{pi}^{CLIP}(\Theta), \ldots, L_{pT}^{CLIP}(\Theta)$, $L_{pi}^{VF}(\Theta), \ldots L_{pT}^{VF}(\Theta)$, $S_{pi}(\pi_\Theta), \ldots S_{pT}(\pi_\Theta)$ in each instance
13: Aggregate losses $L^{total}(\Theta) = \sum_{p=1}^{n}\sum_{i=1}^{T}(L_{pi}^{CLIP} - c_1 L_{pi}^{VF} + c_2 S_{pi}(\pi_\Theta))$
14: Update all parameters,$\Theta_{new} \leftarrow \Theta_{old} + \eta \nabla_\Theta L^{total}(\Theta)$ $L^{total}(\Theta) = \sum_{i=1}^{n}\sum_{p=1}^{T} L_p^i(\Theta)$
15: **end for**
16: **end for**
**End**

pseudocode for the training process. After all the models are trained, we apply 5 models to solve the instances in corresponding size. In order to further test the generalization performance of the model, the model trained on the dataset on j30 and j90 will be directly used to solve instances of other sizes.

To speed up training, the model is trained on the basis the of pre-training model on imitation learning (training for 50 iterations). We set up 8 parallel environments to sample trajectories simultaneously and updated the model parameters 4 times using the importance sampling method [71] in each iteration, the initial learning rate is small and gradually reduced to 0 as training progressed. After training was completed, we checked the average makespan on the validation dataset, took out the model with the best performance, and tested it on testing dataset. There are two methods in testing model: 1. Run our algorithm on the instances once and greedily select actions according to the action probability distribution. 2. Run our algorithm on the instances multiple times, randomly sample action according to the action probability distribution, then record the best scheduling timetable. We compared the differences between the two methods in terms of average makespan and time consumption. Fig. 6(a) a shows that the sample method achieves better results than greedy selection, and when the number of sample times exceeds 30, the improvement is not significant. Therefore, we performed 30 sample times on all instances during testing. Fig. 6(b) shows that the number of sample times is nearly linearly related to time consumption.

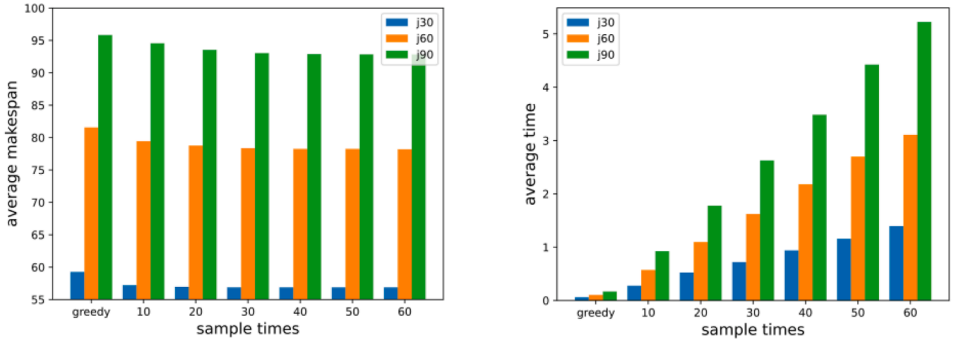
During training, we also tested three different methods for obtaining instances from the training dataset to form a sub-training set for each update. These methods included: 1. Randomly selecting instances from the training dataset; 2. Sequentially selecting instances from the training dataset; and 3. Sequentially selecting instances from training dataset but sample trajectories on each instance for 50 times. The experimental results is shown in Fig. 7. We observed that using method 2 resulted in a more stable and gradual decline in the action loss, and the final result was also better than the other two methods. Therefore, method 2 was used in all subsequent experiments.

In addition, we observed that using batch normalization layers in GNN, compared to not using them, resulted in a decrease in convergence speed and final model performance. This phenomenon was consistent with the description in [72] that it is not necessary to use batch normalization in RL. Therefore, our model does not include batch normalization layers. We also found that as the number of instances in the training set increases, the model performance will improve to some extent. However, after more than 120 instances, it will no longer have a significant impact on model performance, as shown in Fig. 8.

There are hundreds of priority rules proposed for RCPSP, we selected 4 high-performing priority rule algorithms from those reported in [73] for comparison. Although our algorithm adopts a similar approach to priority algorithms, it is not enough to compare it only with priority algorithms. Therefore, a priority rule-based genetic algorithm proposed in [74] and runs for 1000 iterations was also used for comparison. All comparison algorithms are:

1. Latest finishing time (LFT)
2. Latest starting time (LST)
3. Most total successors (MTS)
4. Greatest rank positional weight (GRPW)
5. Genetic algorithm (GA)

The optimal solutions reported in PSPLIB was used as baseline. Gaps between comparison algorithms and baseline are computed by Eq. (13).




(a)The relationship between sample time and makespan (b) The relationship between sample time and time consuming

**Fig. 6.** The relationship between sample time, makespan and time consuming.

$$\%Gap = \frac{Average\ \ makespan\ \ of\ \ comparison\ \ algorilthm - Average\ \ makespan\ \ of\ \ baseline}{Average\ \ makespan\ \ of\ \ baseline} \times 100\% \tag{13}$$

Training curve of the model is shown in Fig. 9. Table 2 shows all the experimental results in RCPSP.

We first evaluated the performance of the model when the training and test sets are of the same size. Our model can be directly applied to unseen instances after training on the training dataset. As the instance size increases, the performance of all algorithms deteriorates, while our algorithm consistently outperforms all priority rules but is inferior to GA. Although our model performs well on j30, j60, and j90, there is a sudden deterioration in performance on j120, while GA's deterioration is relatively small. This implies that our model is better at handling small instances. An investigation of the scheduling results for j30 found that, due to our use of an improved version of MDP, the optimal solution achievable by the environment on some instances (such as instance 4, 20, and 22) has a small gap with the optimal solution reported in PSPLIB. Our model schedules 69 out of 90 optimal solutions achievable by the environment. Finally, in terms of computation time, our algorithm did not show a significant increase compared to GA as the instance size increased.

We further evaluated the generalization ability of our model. Results on the mixed dataset indicate that our model can be trained directly on mixed instances of different sizes and used to solve mixed instances. For models trained only on j30 and j60 instances, they can be directly applied to solve other size instances, but their performance is inferior compared to models trained and tested on the same size. By comparing these two models, we found that the latter performs better on other instances. This result suggests that our approach can extract knowledge from small size instances, and larger size instances enable the model to learn better scheduling strategies.

### 5.3. Training and testing on RCPSP-RD

In this section, we do not make any changes to the model structure. All parameter settings follow the results of previous discussions. The only change is the training environment described in Section 4.2, we train the model and test its results on RCPSP-RD.

Since PSPLIB does not include RCPSP with uncertain resource capacity, we expanded the PSPLIB dataset using the same method as [57]. The deterministic resource capacity in RCPSP was replaced by random resource capacity with different probability distribution, to simulate different types of the resource capacity conditions in reality. We constructed the uniform, beta and exponential distributions, and used the original resource availability $R_k$ in PSPLIB instances as the mean of the uncertain resource capacity. A lower bound of resource capacity in each instance was set to $LR_k = \max_{j \in V}\{r_{jk}\}$, to prevent the instance from

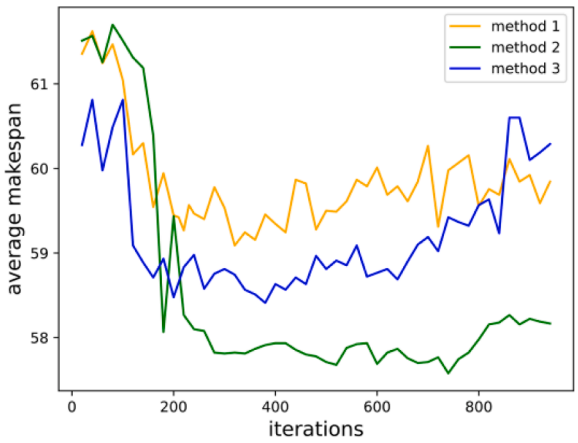



**Fig. 7.** The relationship between different obtain instances methods and average makespan on validation dataset on j30.

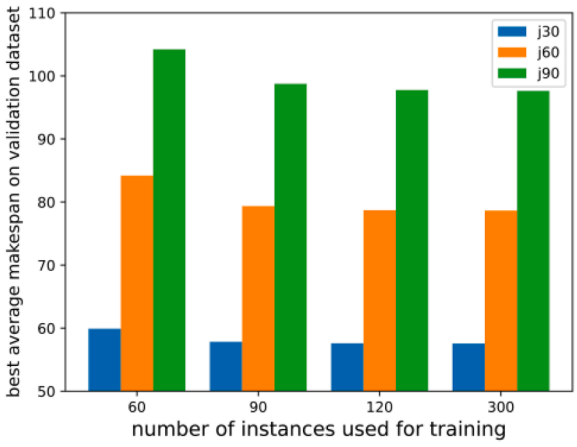



**Fig. 8.** The relationship between number of instances for training and best average makespan on validation dataset.

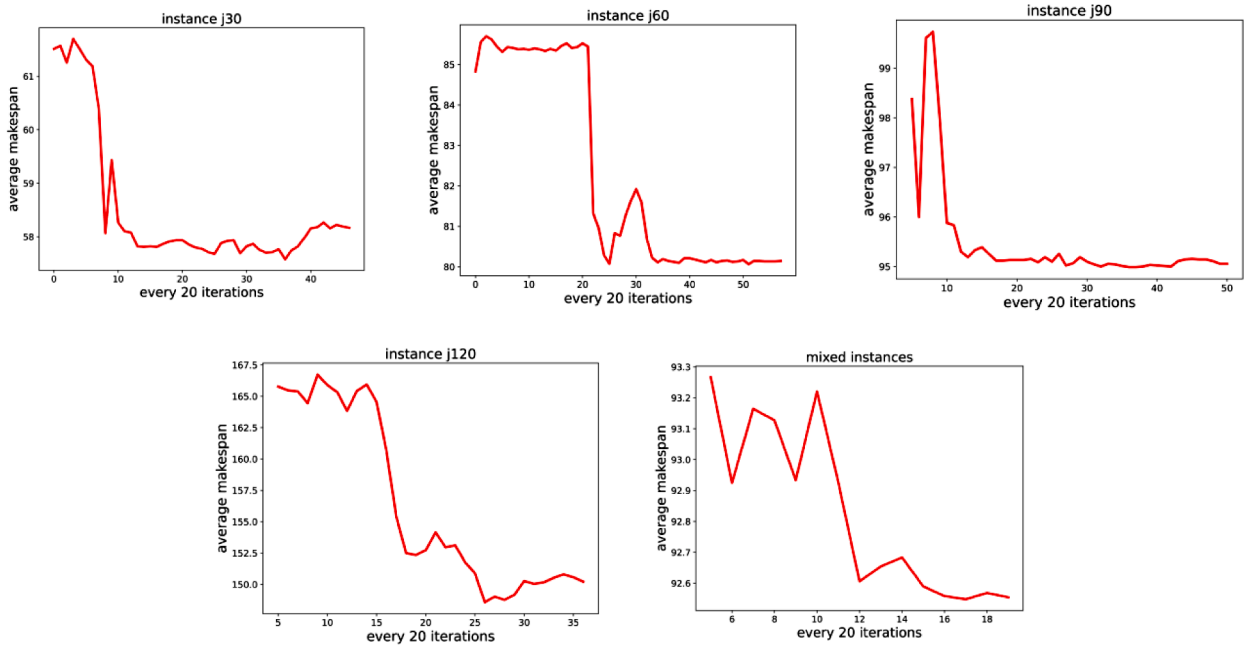



**Fig. 9.** Training curves of RCPSPs with different size.

**Table 2**
Comparison of different algorithm in RCPSP.

| Size | | LFT | LST | GRPW | MTS | GA | Ours | Ours (j30) | Ours (j90) |
|---|---|---|---|---|---|---|---|---|---|
| J30 | makespan | 65.22 | 68.3 | 73.15 | 71.35 | 55.75 | 56.97 | • | 58.07 |
| | Gap | 18.17% | 23.75% | 32.54% | 29.28% | 1.01% | 3.22% | • | 0.0497 |
| | Time(s) | 0.025 | 0.022 | 0.0018 | 0.0017 | 2.1566 | 0.7928 | • | 0.6438 |
| J60 | makespan | 92.53 | 92.32 | 103.71 | 98.87 | 75.98 | 78.32 | 81.13 | 79.82 |
| | Gap | 25.15% | 24.87% | 40.28% | 33.73% | 2.77% | 5.91% | 9.74% | 7.38% |
| | Time(s) | 0.083 | 0.0746 | 0.0048 | 0.0043 | 5.1566 | 1.1266 | 1.1162 | 1.1192 |
| J90 | makespan | 110.53 | 116.25 | 122.88 | 122.52 | 90.06 | 93.05 | 95.43 | • |
| | Gap | 28.89% | 35.56% | 43.30% | 42.88% | 5.02% | 8.51% | 11.29% | • |
| | Time(s) | 0.1891 | 0.1651 | 0.0088 | 0.0082 | 11.6518 | 2.4193 | 2.4063 | • |
| J120 | makespan | 146.8 | 150.55 | 162.2 | 158.31 | 103.61 | 113.2 | 117.56 | • |
| | Gap | 49.68% | 53.51% | 65.39% | 61.42% | 5.64% | 15.42% | 19.88% | • |
| | Time(s) | 0.3471 | 0.301 | 0.0146 | 0.0133 | 28.2851 | 4.046 | 4.012 | • |
| Mixd | makespan | 106.27 | 110.1 | 122.23 | 117.01 | 82.4 | 85.65 | 88.00 | 87.11 |
| | Gap | 35.82% | 40.72% | 56.22% | 49.55% | 5.31% | 9.47% | 12.48% | 10.18% |
| | Time(s) | 0.1423 | 0.1423 | 0.0075 | 0.0068 | 8.1849 | 1.9696 | 1.8616 | 1.8911 |

**Table 3**
Three different distributions in RCPSP-RD.

| Type | Support | Variance | Symmetric |
|---|---|---|---|
| Uniform | $[LR_k, 2R_k - LR_k]$ | $(R_k - LR_k)^2/3$ | Yes |
| Beta | $[LR_k, 2R_k - LR_k]$ | $(R_k - LR_k)^2/3$ | No |
| Exponential | – | $\approx R_k^2$ | No |

becoming unsolvable, as shown in Table 3.

The variance of the exponential distribution is larger than uniform and beta distribution. $\alpha$ and $\beta$ in beta distribution were set to 1. For the exponential distribution, we first generated the resources with $R_k$ as the expectation and then replace $R_k$ with $LR_k$ for all cases of $LR_k > R_k$, so that the actual variance will be slightly less than $R_k^2$. The resource capacities of all instances from 0 to 1000th-time point were generated according to these distributions.

The division of the datasets, training and testing process, was essentially the same as that in Section 5.2. For instances of different sizes from the three distributions, we trained a separate model for each distribution and a corresponding model for each mixed dataset. The generalization ability of the models was also tested as the described method above.

In terms of comparison algorithms, the four priority algorithms introduced in Section 5.2 would continue to be used as comparison algorithms. However, the genetic algorithm described in that section can no longer be used, we will replace it with an ADP algorithm described in [57], which be reported to outperforms many heuristics in SRCPSP. Since the optimal solution is unknown, the ADP algorithm is used as the baseline and the gap between it and all other algorithms was computed. All training curves are shown in Figs. 10–12. Table 4, Table 5 and Table 6, respectively show the test results of all algorithms on uniform, beta, and exponential datasets.

We first evaluate those models with the same size of training and test datasets. Our model also has generalization ability on RCPSP-RD and can be directly used to process new instances after training. In terms of performance, although our algorithm cannot surpass ADP in makespan, it still leads all priority algorithms by a large margin. Our algorithm did

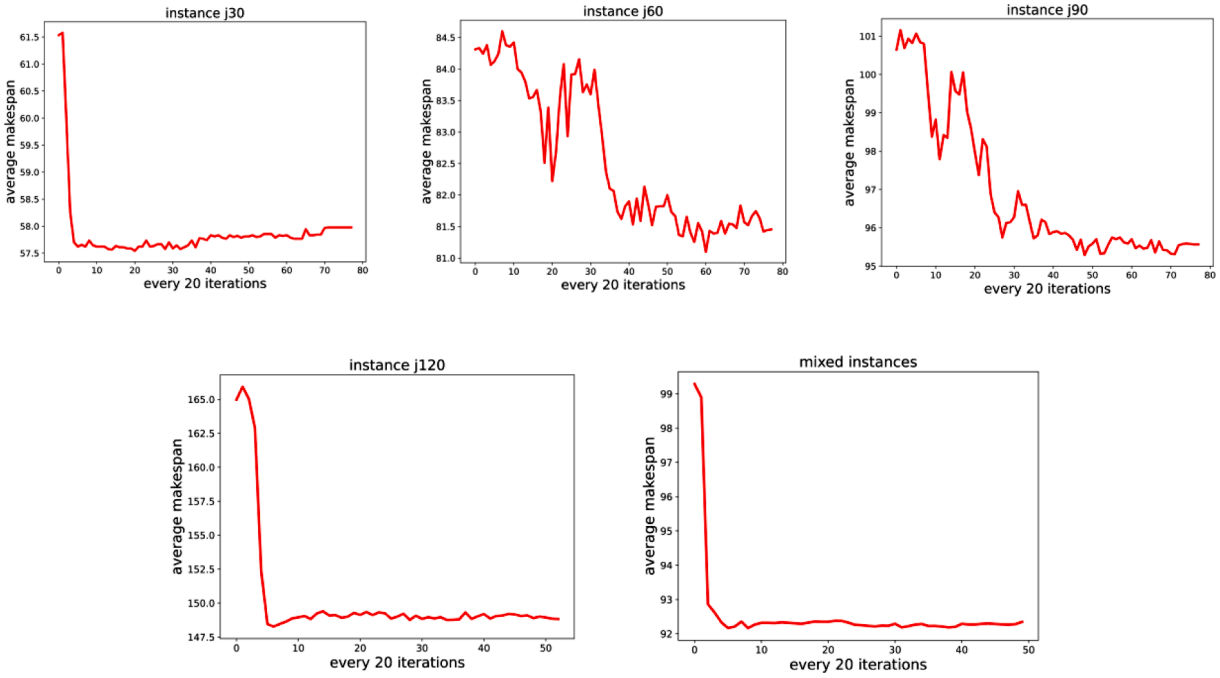



**Fig. 10.** Training curves of RCPSP-RD, uniform distribution.

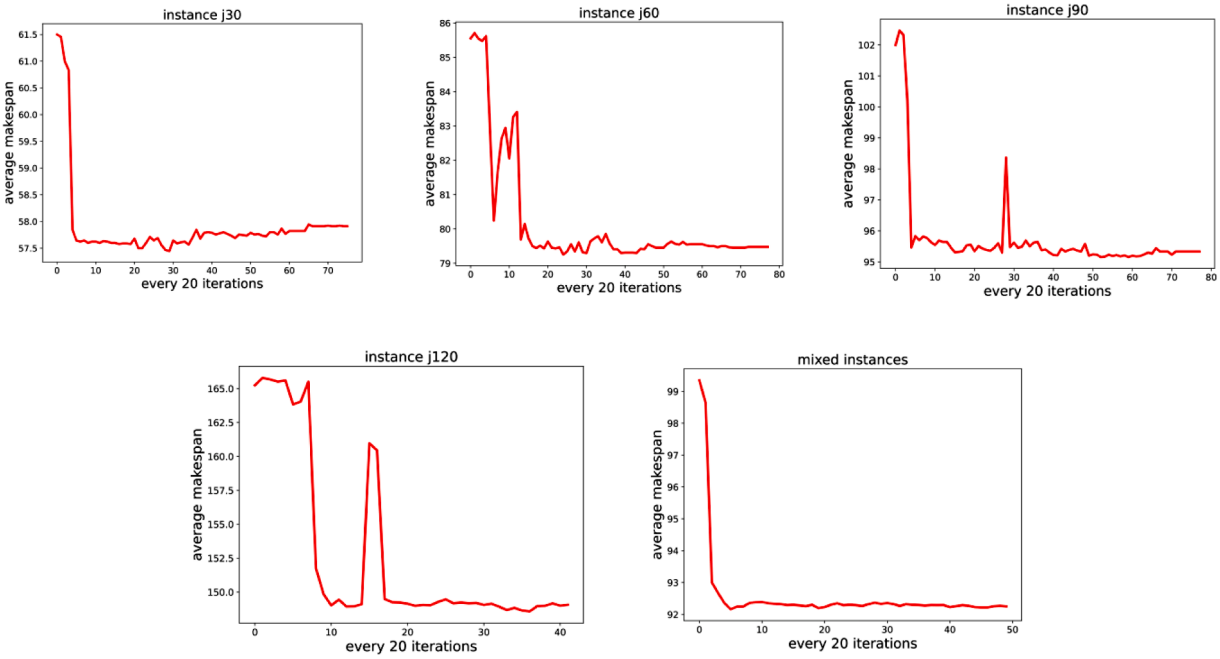



**Fig. 11.** Training curves of RCPSP-RD, beta distribution.

not widen the gap with the ADP algorithm on all types of resource distributions on j30, j60, and j90, but there was a significant performance decline on j120, which was more severe than the decline on RCPSP. Moreover, compared to the other two distributions, the decline in the exponential distribution with larger variance is more severe, indicating that our model is not good at handle instances with larger scale and resource variance. We discussed the reasons for the decline in performance of the model on larger-scale instances in Section 6.

In terms of the generalization ability of the model, it is basically the same as the conclusion obtained by the model in RCPSP. However, compared to the generalization ability performance of the model in RCPSP, the corresponding model in RCPSP-RD has a worse generalization ability on j30 and j90. This phenomenon indicates that it is more difficult for the model to learn general scheduling rules on dynamic problems. It is worth mentioning that the calculation time required by the ADP algorithm increases significantly as the instance size increases, while ours does not change much. Overall, our algorithm did not widen the gap with the ADP algorithm on j30, but exchanged a large amount of time for a small loss of performance on j60 and j90, so our algorithm is competitive.

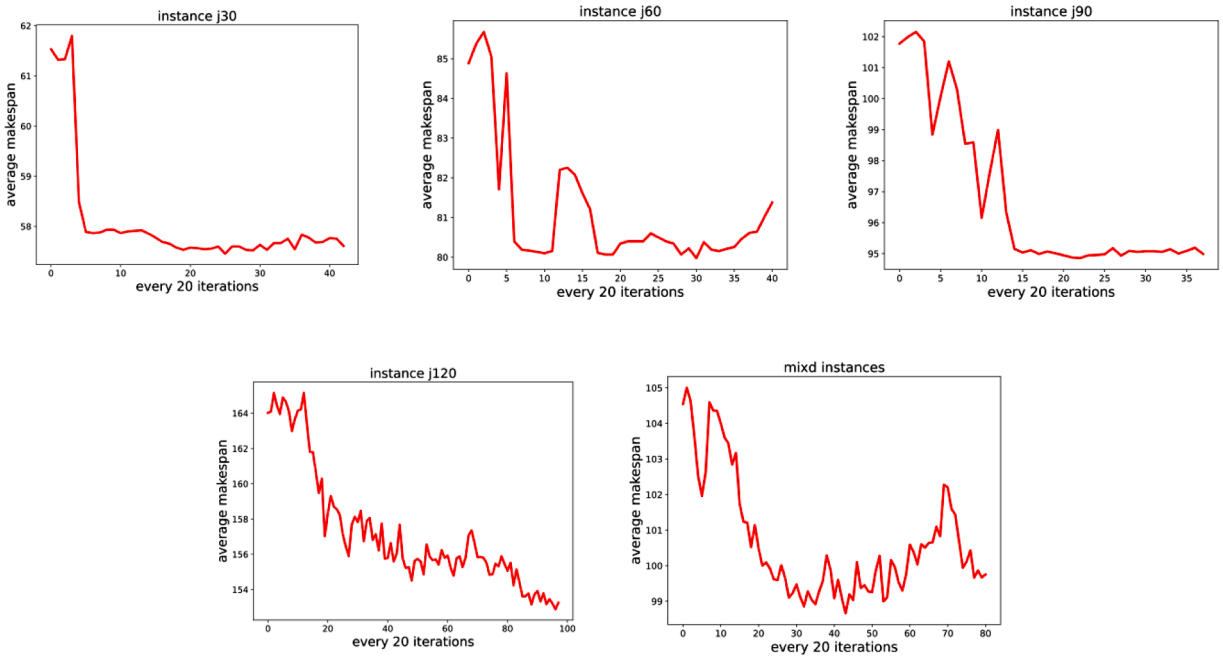



**Fig. 12.** Training curves of RCPSP-RD, exponential distribution.

**Table 4**
Comparison of different algorithm in RCPSP-RD with uniform distribution.

| Uniform distribution | | LFT | LST | GRPW | MTS | ADP | Ours | Ours (j30) | Ours (j90) |
|---|---|---|---|---|---|---|---|---|---|
| J30 | makespan | 66.26 | 68.41 | 73.42 | 70.8 | 56.62 | 57.94 | • | 59.21 |
| | Gap | 17.02% | 20.82% | 29.67% | 25.04% | • | 2.33% | • | 4.57% |
| | Time(s) | 0.0616 | 0.0514 | 0.0044 | 0.0049 | 14.9021 | 1.8304 | • | 1.8519 |
| J60 | makespan | 98.82 | 99.25 | 118.23 | 113.36 | 78.28 | 81.91 | 84.16 | 82.51 |
| | Gap | 26.23% | 26.78% | 51.03% | 44.81% | • | 4.63% | 7.51% | 5.40% |
| | Time(s) | 0.2247 | 0.1842 | 0.0105 | 0.0108 | 162.6216 | 4.9865 | 4.8119 | 4.9218 |
| J90 | makespan | 122.81 | 125.22 | 143.56 | 135.83 | 91.03 | 98.95 | 103.16 | • |
| | Gap | 34.91% | 37.54% | 57.70% | 49.21% | • | 8.70% | 13.32% | • |
| | Time(s) | 0.5236 | 0.4153 | 0.0158 | 0.0174 | 752.1896 | 8.3217 | 8.9217 | • |
| J120 | makespan | 162.4 | 165.26 | 179.04 | 172.85 | 103.48 | 120.04 | 131.18 | • |
| | Gap | 56.93% | 59.70% | 73.01% | 67.03% | • | 16.00% | 26.76% | • |
| | Time(s) | 0.9915 | 0.8003 | 0.024 | 0.0268 | 2701.182 | 13.55 | 14.24 | • |
| Mixd | makespan | 114.07 | 117.28 | 135.06 | 125.21 | 81.45 | 89.59 | 94.19 | 93.65 |
| | Gap | 40.04% | 43.99% | 65.81% | 53.72% | • | 9.99% | 15.64% | 14.97% |
| | Time(s) | 0.3962 | 0.3666 | 0.0141 | 0.015 | 613.1899 | 7.194 | 7.1123 | 7.2101 |

## 6. Discussions

We discussed why our algorithm performs well on most instances but was not good at handling j120. We speculate that this is related to the limitations of GNN. The number of layers in a GNN determines the range a node can observe. As the graph grows larger, the hierarchy of the graph becomes deeper, and the information that needs to be considered for decision-making also increases. Therefore, for each schedulable node, its receptive field needs to be correspondingly increased to take into account more information from surrounding nodes. In imitation learning experiments, we found that as the number of GNN layers increases, the upper limit of the model's learning ability also improves. However, we cannot increase K indefinitely. When K increases, GNN is more likely to suffer from over-smoothing [75], which will cause its feature extraction ability to decline. This is also confirmed in our imitation learning experiment that when K is greater than 5, the upper limit of the model's ability is actually worse than that of a model with fewer layers. In summary, we speculate that the model is not good at handling large-size problems due to the inherent limitations of GNN. However, we believe that with the advancement of GNN technology, this may be resolved in the future.

We think that our algorithm has great development potential in solving practical scheduling problems. In some practical cases, such as aircraft assembling, the resources change in various ways in different assembly stations and are difficult to express. For our algorithm, we can modify part of the process of the MDP according to the specific problem, use imitation learning to pre-train the model, and use reinforcement learning for further training on realistic historical cases. The ability to train directly on real cases enables our algorithm to learn different scheduling strategies according to the characteristics of different scenarios. We think that this ability to learn is what fundamentally differentiates our approaches from traditional approaches.

## 7. Conclusions

This study proposed a framework based on GNN and RL that can automatically learn scheduling strategies to deal with RCPSP and RCPSP-RD. The scheduling process is formulated as sequential decision-

**Table 5**
Comparison of different algorithm in RCPSP-RD with beta distribution.

| Beta distribution | | LFT | LST | GRPW | MTS | ADP | Ours | Ours (j30) | Ours (j90) |
|---|---|---|---|---|---|---|---|---|---|
| J30 | makespan | 65.98 | 69.53 | 74.94 | 72.16 | 57.11 | 58.08 | • | 59.03 |
| | Gap | 15.53% | 21.74% | 31.22% | 26.35% | • | 1.69% | • | 3.36% |
| | Time(s) | 0.0614 | 0.0493 | 0.0046 | 0.0047 | 12.5121 | 1.9506 | • | 1.9878 |
| J60 | makespan | 98.44 | 100.08 | 121.28 | 115.15 | 79.15 | 82.31 | 84.76 | 83.65 |
| | Gap | 24.37% | 26.44% | 53.22% | 45.48% | • | 3.99% | 7.08% | 5.68% |
| | Time(s) | 0.2208 | 0.1876 | 0.0096 | 0.0105 | 158.1648 | 4.9865 | 5.3961 | 5.2198 |
| J90 | makespan | 122.97 | 126.47 | 147.36 | 142.38 | 92.68 | 98.91 | 104.51 | • |
| | Gap | 32.68% | 36.45% | 58.99% | 53.62% | • | 6.72% | 12.76% | • |
| | Time(s) | 0.5347 | 0.419 | 0.0161 | 0.0179 | 737.2618 | 8.8933 | 8.9514 | • |
| J120 | makespan | 163.47 | 165.6 | 184.45 | 175.06 | 104.19 | 120.8 | 134.26 | • |
| | Gap | 56.89% | 58.94% | 77.03% | 68.02% | • | 15.94% | 28.86% | • |
| | Time(s) | 0.9859 | 0.8063 | 0.0239 | 0.0263 | 2630.658 | 14.1872 | 14.5176 | • |
| Mixd | makespan | 112.41 | 119.27 | 138.7 | 128.43 | 87.023 | 93.86 | 91.58 | 83.01 |
| | Gap | 35.41 | 43.68 | 67.08 | 54.71 | 4.83 | 13.07 | 10.32 | • |
| | Time(s) | 0.3654 | 0.3434 | 143.43 | 0.0145 | 6.9385 | 6.7219 | 6.7198 | 599.1845 |

**Table 6**
Comparison of different algorithm in RCPSP-RD with exponential distribution.

| Exponential distribution | | LFT | LST | GRPW | MTS | ADP | Ours | Ours (j30) | Ours (j90) |
|---|---|---|---|---|---|---|---|---|---|
| J30 | makespan | 72.32 | 71.33 | 83.63 | 78.6 | 61.74 | 62.87 | • | 64.68 |
| | Gap | 17.13% | 15.53% | 35.45% | 27.30% | • | 1.83% | • | 4.76% |
| | Time(s) | 0.0862 | 0.0761 | 0.0063 | 0.0063 | 17.1328 | 2.6625 | • | 2.9317 |
| J60 | makespan | 119.77 | 118.22 | 136.65 | 131.33 | 88.12 | 92.16 | 95.61 | 95.12 |
| | Gap | 35.91% | 34.15% | 55.07% | 49.03% | • | 4.58% | 8.49% | 7.94% |
| | Time(s) | 0.2995 | 0.2514 | 0.0128 | 0.0139 | 223.2981 | 6.4081 | 6.3212 | 6.4132 |
| J90 | makespan | 156.95 | 160.18 | 179.14 | 173.02 | 105.59 | 112.53 | 118.16 | • |
| | Gap | 48.64% | 51.70% | 69.65% | 63.86% | • | 6.57% | 11.90% | • |
| | Time(s) | 0.673 | 0.5689 | 0.0206 | 0.0223 | 842.2554 | 12.6787 | 11.8618 | • |
| J120 | makespan | 172.86 | 175.9 | 203.53 | 190.07 | 111.19 | 131.72 | 144.16 | 138.15 |
| | Gap | 55.46% | 58.19% | 83.04% | 70.94% | . | 18.46% | 29.65% | 24.24% |
| | Time(s) | 1.2586 | 1.064 | 0.0302 | 0.0323 | 3015.188 | 15.8052 | 16.8182 | 15.8052 |
| Mixd | makespan | 124.23 | 130.63 | 153.27 | 136.21 | 85.79 | 91.85 | 97.42 | 96.13 |
| | Gap | 44.80% | 52.26% | 78.65% | 58.77% | • | 7.06% | 13.55% | 12.05% |
| | Time(s) | 0.4974 | 0.473 | 0.0168 | 0.0182 | 723.2198 | 8.6977 | 10.6977 | 10.3548 |

making problems, then the corresponding MDP model is formulated. A GNN-based structure is proposed to extract features from problems and then make decisions. PPO is used to train the model in an end-to-end way to optimize the performance of the model. The impact of parameters on the performance of the model and the performance of the model is analyzed. In conclusion, the proposed framework achieved competitive results compared to some commonly used algorithms.

Further studies can be carried out in the following areas. 1. Further improve model performance by improving the model structure and reward function, especially in combination with recurrent neural networks, such as using recurrent neural networks to extract resource change information and incorporating it into the decision-making process. GNNs that are better at capturing long-range interactions between nodes also need to be further studied. 2. Extend this method to other types of static and dynamic RCPSP problems, such as multi-objectives, multi-execution modes, new project arrivals, etc.

## CRediT authorship contribution statement

**Hongxia Cai:** Conceptualization, Methodology, Resources, Investigation, Supervision, Writing – original draft, Writing – review & editing. **Yunqi Bian:** Conceptualization, Methodology, Formal analysis, Investigation, Software, Validation, Writing – original draft. **Lilan Liu:** Conceptualization, Methodology, Supervision, Funding acquisition.

## Declaration of Competing Interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Hongxia Cai, Yunqi Bian, Lilan Liu reports financial support was provided by National Key Research and Development Program of China.

## Data availability

The code for this paper is available in https://github.com/Frag-Ment/RG-RRD

## Acknowledgments

The authors are grateful for financial support from the National Key Research and Development Program of China (No. 2021YFB3300503).

## References

[1] E.L. Demeulemeester, W.S. Herroelen. Project scheduling: a research handbook, Springer Science & Business Media, 2006.
[2] P. Brucker, Scheduling and constraint propagation, Discret. Appl. Math. 123 (2002) 227–256, https://doi.org/10.1016/s0166-218x(01)00342-0.

[3] W.N. Chen, J. Zhang, Ant colony optimization for software project scheduling and staffing with an event-based scheduler, IEEE Trans. Softw. Eng. 39 (2013) 1–17, https://doi.org/10.1109/tse.2012.17.
[4] S.Q. Shan, Z.J. Hu, Z.L. Liu, J.H. Shi, L. Wang, Z.M. Bi, An adaptive genetic algorithm for demand-driven and resource-constrained project scheduling in aircraft assembly, Inf. Technol. Manag. 18 (2017) 41–53, https://doi.org/10.1007/s10799-015-0223-7.
[5] J.L. Wang, J. Zhang, X.X. Wang, A data driven cycle time prediction with feature selection in a semiconductor wafer fabrication system, IEEE Trans. Semicond. Manuf. 31 (2018) 173–182, https://doi.org/10.1109/tsm.2017.2788501.
[6] M. Bartusch, R.H. Mhring, F.J. Radermacher, Scheduling project networks with resource constraints and time windows, Ann. Oper. Res. 16 (1988) 199–240, https://doi.org/10.1007/BF02283745.
[7] A. Pritsker, L.J. Waiters, P.M. Wolfe, Multiproject Scheduling with Limited Resources: A Zero-One Programming Approach, INFORMS, 1969, https://doi.org/10.1287/mnsc.16.1.93.
[8] A. Mingozzi, V. Maniezzo, S. Ricciardelli, L. Bianco, An exact algorithm for the resource-constrained project scheduling problem based on a new mathematical formulation, Manag. Sci. 44 (1998) 714–729, https://doi.org/10.1287/mnsc.44.5.714.
[9] A. Sprecher, Scheduling resource-constrained projects competitively at modest memory requirements, Manag. Sci. 46 (2000), https://doi.org/10.1287/mnsc.46.5.710.12044.
[10] R. Kolisch, S. Hartmann, Heuristic algorithms for solving the resource-constrained project scheduling problem: classification and computational analysis, Manuskripte aus den Instituten für Betriebswirtschaftslehre der Universität Kiel, (1998). 10.1007/978-1-4615-5533-9_7.
[11] R. Kolisch, S. Hartmann, Experimental investigation of heuristics for resource-constrained project scheduling: an update, Eur. J. Oper. Res. 174 (2006) 23–37, https://doi.org/10.1016/j.ejor.2005.01.065.
[12] L. Bianco, M. Caramia, A new lower bound for the resource-constrained project scheduling problem with generalized precedence relations, Comput. Oper. Res. 38 (2011) 14–20, https://doi.org/10.1016/j.cor.2009.07.003.
[13] P. Brucker, A. Drexl, R. M?Hring, K. Neumann, E. Pesch, Resource-constrained project scheduling: notation, classification, models, and methods, Eur. J. Oper. Res. 112 (1999) 3–41, https://doi.org/10.1016/S0377-2217(98)00204-5.
[14] A. Kimms, Maximizing the net present value of a project under resource constraints using a lagrangian relaxation based heuristic with tight upper bounds, Ann. Oper. Res. 102 (2001) 221–236, https://doi.org/10.1023/A:1010962300979.
[15] F. Talbot, Resource-constrained project scheduling with time-resource tradeoffs: the nonpreemptive case, Manag. Sci. 28 (1982) 1197–1210, https://doi.org/10.1287/mnsc.28.10.1197.
[16] Y. Cheng, Y. Zhang, P. Ji, W. Xu, Z. Zhou, F. Tao, Cyber-physical integration for moving digital factories forward towards smart manufacturing: a survey, Int. J. Adv. Manuf. Technol. 97 (2018) 1209–1221, https://doi.org/10.1007/s00170-018-2001-2.
[17] E. Oztemel, S. Gursev, Literature review of Industry 4.0 and related technologies, J. Intell. Manuf. 31 (2020) 127–182, https://doi.org/10.1007/s10845-018-1433-8.
[18] O. Lambrechts, E. Demeulemeester, W. Herroelen, Time slack-based techniques for robust project scheduling subject to resource uncertainty, Ann Oper Res 186 (2011) 443–464, https://doi.org/10.1007/s10479-010-0777-z.
[19] B. Cunha, A.M. Madureira, B. Fonseca, D. Coelho, Deep Reinforcement Learning as a Job Shop Scheduling Solver: A Literature Review, Springer, Cham, 2018, https://doi.org/10.1007/978-3-030-14347-3_34.
[20] J. Park, J. Chun, S.H. Kim, Y. Kim, J. Park, Learning to schedule job-shop problems: representation and policy learning using graph neural network and reinforcement learning, Int. J. Prod. Res. 59 (2021) 3360–3377, https://doi.org/10.1080/00207543.2020.1870013.
[21] M. Hameed, A. Schwung, Reinforcement learning on job shop scheduling problems using graph networks, (2020). 10.13140/RG.2.2.13862.96326.
[22] H. Mao, M. Alizadeh, I. Menache, S. Kandula, Resource management with deep reinforcement learning, in: Proceedings of the 15th ACM Workshop on Hot Topics in Networks, 2016, pp. 50–56.
[23] C. Zhang, W. Song, Z. Cao, J. Zhang, P.S. Tan, X. Chi, Learning to Dispatch for Job Shop Scheduling via Deep Reinforcement Learning, in: Advances in Neural Information Processing Systems, Curran Associates, Inc, 2020, pp. 1621–1632.
[24] J. Choi, M.J. Realff, J.H. Lee, A Q-Learning-based method applied to stochastic resource constrained project scheduling with new project arrivals, Int. J. Robust Nonlinear Control 17 (2010), https://doi.org/10.1002/rnc.1164.
[25] P. Jedrzejowicz, E. Ratajczak-Ropel, Reinforcement learning strategies for A-team solving the resource-constrained project scheduling problem, Neurocomputing 146 (2014) 301–307, https://doi.org/10.1016/j.neucom.2014.05.070.
[26] P. Jędrzejowicz, E. Ratajczak-Ropel, Reinforcement Learning Strategy for Solving the MRCPSP by a Team of Agents, Springer International Publishing, 2015.
[27] K.M. Sallam, R.K. Chakrabortty, M.J. Ryan, A reinforcement learning based multi-method approach for stochastic resource constrained project scheduling problems, Expert Syst. Appl. 169 (2021) 19, https://doi.org/10.1016/j.eswa.2020.114479.
[28] J. Shahrabi, M.A. Adibi, M. Mahootchi, A reinforcement learning approach to parameter estimation in dynamic job shop scheduling, Comput. Ind. Eng. 110 (2017) 75–82, https://doi.org/10.1016/j.cie.2017.05.026.
[29] Y.X. Li, W.N. Gu, M.H. Yuan, Y.M. Tang, Real-time data-driven dynamic scheduling for flexible job shop with insufficient transportation resources using hybrid deep Q network, Robot. Comput. Integr. Manuf. 74 (2022) 16, https://doi.org/10.1016/j.rcim.2021.102283.
[30] X.H. Wang, L. Zhang, T.Y. Lin, C. Zhao, K.Y. Wang, Z. Chen, Solving job scheduling problems in a resource preemption environment with multi-agent reinforcement learning, Robot. Comput. Integr. Manuf. 77 (2022) 13, https://doi.org/10.1016/j.rcim.2022.102324.
[31] Y. Zhang, H. Zhu, D. Tang, T. Zhou, Y. Gui, Dynamic job shop scheduling based on deep reinforcement learning for multi-agent manufacturing systems, Robot. Comput. Integr. Manuf. 78 (2022), 102412, https://doi.org/10.1016/j.rcim.2022.102412.
[32] Y. Liu, J. Fan, L. Zhao, W. Shen, C. Zhang, Integration of deep reinforcement learning and multi-agent system for dynamic scheduling of re-entrant hybrid flow shop considering worker fatigue and skill levels, Robot. Comput. Integr. Manuf. 84 (2023), 102605, https://doi.org/10.1016/j.rcim.2023.102605.
[33] H. Mao, M. Schwarzkopf, S.B. Venkatakrishnan, Z. Meng, M. Alizadeh, Learning Scheduling Algorithms for Data Processing Clusters, The ACM Special Interest Group, 2019.
[34] E. Yuan, S. Cheng, L. Wang, S. Song, F. Wu, Solving job shop scheduling problems via deep reinforcement learning, Appl. Soft Comput. (2023), 110436, https://doi.org/10.1016/j.asoc.2023.110436.
[35] Y.J. Hu, Y. Yao, W.S. Lee, A reinforcement learning approach for optimizing multiple traveling salesman problems over graphs, Knowl. Based Syst. 204 (2020) 14, https://doi.org/10.1016/j.knosys.2020.106244.
[36] O. Lambrechts, E. Demeulemeester, W. Herroelen, Proactive and reactive strategies for resource-constrained project scheduling with uncertain resource availabilities, J. Sched. 11 (2008) 121–136, https://doi.org/10.1007/s10951-007-0021-0.
[37] W. Herroelen, R. Leus, Robust and reactive project scheduling: a review and classification of procedures, Int. J. Prod. Res. 42 (2004), https://doi.org/10.1080/00207540310001638055.
[38] E.M. Goldratt, Critical Chain, Critical Chain, 1997.
[39] M. Davari, E. Demeulemeester, The proactive and reactive resource-constrained project scheduling problem, J. Sched. 22 (2019) 211–237, https://doi.org/10.1007/s10951-017-0553-x.
[40] M. Brcic, M. Katic, N. Hlupic, Planning horizons based proactive rescheduling for stochastic resource-constrained project scheduling problems, Eur. J. Oper. Res. 273 (2019) 58–66, https://doi.org/10.1016/j.ejor.2018.07.037.
[41] Q.G. Bai, J.T. Xu, Y.Z. Zhang, The distributionally robust optimization model for a remanufacturing system under cap-and-trade policy: a newsvendor approach, Ann. Oper. Res. 309 (2022) 731–760, https://doi.org/10.1007/s10479-020-03642-4.
[42] G. Igelmund, F.J. Radermacher, Preselective strategies for the optimization of stochastic project networks under resource constraints, Networks 13 (2010) 1–28, https://doi.org/10.1002/net.3230130102.
[43] H.T. Li, N.K. Womer, Solving stochastic resource-constrained project scheduling problems by closed-loop approximate dynamic programming, Eur. J. Oper. Res. 246 (2015) 20–33, https://doi.org/10.1016/j.ejor.2015.04.015.
[44] R.L. Graham, Bounds on multiprocessing timing anomalies, SIAM J. Appl. Math. 17 (1969) 416–429, https://doi.org/10.1137/0117039.
[45] G. Lgelmund, F.J. Radermacher, Algorithmic approaches to preselective strategies for stochastic scheduling problems, Networks 13 (2010) 29–48, https://doi.org/10.1002/net.3230130103.
[46] D. GolenkoGinzburg, A. Gonik, Stochastic network project scheduling with non-consumable limited resources, Int. J. Prod. Econ. 48 (1997) 29–37, https://doi.org/10.1016/s0925-5273(96)00019-9.
[47] M. Gendreau, J.-Y. Potvin. Handbook of metaheuristics, Springer, 2010.
[48] Y.W. Tsai, D.D. Gemmill, Using tabu search to schedule activities of stochastic resource-constrained projects, Eur. J. Oper. Res. 111 (1998) 129–141, https://doi.org/10.1016/s0377-2217(97)00311-1.
[49] F. Ballesti, When it is worthwhile to work with the stochastic RCPSP? J. Sched. 10 (2007) 153–166, https://doi.org/10.1007/s10951-007-0012-1.
[50] B. Ashtiani, R. Leus, M.B. Aryanezhad, New competitive results for the stochastic resource-constrained project scheduling problem: exploring the benefits of pre-processing, J. Sched. 14 (2011) 157–171, https://doi.org/10.1007/s10951-009-0143-7.
[51] S. Rostami, S. Creemers, R. Leus, New strategies for stochastic resource-constrained project scheduling, J. Sched. (2017), https://doi.org/10.1007/s10951-016-0505-x.
[52] S.E. Dreyfus, A.M. Law, The Art and Theory of Dynamic Programming, Academic Press, Inc., 2014, https://doi.org/10.2307/3009654.
[53] S. Ross, Introduction to Stochastic Dynamic Programming, Academic Press, 1983, https://doi.org/10.2307/2289262.
[54] S. Fianu, L.B. Davis, A Markov decision process model for equitable distribution of supplies under uncertainty, Eur. J. Oper. Res. (2018), https://doi.org/10.1016/j.ejor.2017.07.017. S0377221717306446.
[55] L.A. Baxter, Markov decision processes: discrete stochastic dynamic programming, Technometrics 37 (1995), https://doi.org/10.1080/00401706.1995.10484354, 353-353.
[56] J. Choi, M.J. Realff, J.H. Lee, Dynamic programming in a heuristically confined state space: a stochastic resource-constrained project scheduling application, Comput. Chem. Eng. 28 (2004) 1039–1058, https://doi.org/10.1016/j.compchemeng.2003.09.024.
[57] F. Xie, H.T. Li, Z. Xu, An approximate dynamic programming approach to project scheduling with uncertain resource availabilities, Appl. Math. Model. 97 (2021) 226–243, https://doi.org/10.1016/j.apm.2021.03.048.
[58] R. Alvarez-Valdes, J.M. Tamarit, Heuristic algorithms for resource-constrained project scheduling: a review and an empirical analysis in R, heuristic algorithms for resource-constrained project scheduling: a review and an empirical analysis in R, 1989.
[59] P.W. Battaglia, J.B. Hamrick, V. Bapst, A. Sanchez-Gonzalez, V. Zambaldi, M. Malinowski, A. Tacchetti, D. Raposo, A. Santoro, R. Faulkner, Relational inductive biases, deep learning, and graph networks, (2018). 10.48550/arXiv.1806.01261.

[60] J. Zhou, G. Cui, Z. Zhang, C. Yang, Z. Liu, L. Wang, C. Li, M. Sun, Graph neural networks: a review of methods and applications, (2018). 10.48550/arXiv.1812.08434.
[61] M. Lv, Z. Hong, L. Chen, T. Chen, S. Ji, Temporal multi-graph convolutional network for traffic flow prediction, IEEE Trans. Intell. Transp. Syst. (2020) 1–12, https://doi.org/10.1109/TITS.2020.2983763.
[62] K. Xu, W. Hu, J. Leskovec, S. Jegelka, How powerful are graph neural networks?, in, 2018.
[63] L. Wang, X. Hu, Y. Wang, S. Xu, S. Ma, K. Yang, Z. Liu, W. Wang, Dynamic job-shop scheduling in smart manufacturing using deep reinforcement learning, Comput. Netw. 190 (2021), https://doi.org/10.1016/j.comnet.2021.107969.
[64] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal Policy Optimization Algorithms, (2017). 10.48550/arXiv.1707.06347.
[65] J. Schulman, S. Levine, P. Moritz, M.I. Jordan, P. Abbeel, Trust region policy optimization, Comput. Sci. (2015) 1889–1897, https://doi.org/10.48550/arXiv.1502.05477.
[66] R. Kolisch, A. Sprecher, PSPLIB – a project scheduling problem library, Eur. J. Oper. Res. 96 (1997) 205–216, https://doi.org/10.1016/S0377-2217(96)00170-1.
[67] R. Kolisch, S.A. Drexl, Characterization and generation of a general class of resource-constrained project scheduling problems, Manag. Sci. 41 (1995) 1693–1703, https://doi.org/10.2307/2632747.
[68] A. Hussein, M.M. Gaber, E. Elyan, C. Jayne, Imitation learning: a survey of learning methods, ACM Comput. Surv. 50 (2017) 1–35, https://doi.org/10.1145/3054912 (CSUR).
[69] S. Huang, R.F.J. Dossa, C. Ye, J. Braga, D. Chakraborty, K. Mehta, J.G. Araújo, Cleanrl: high-quality single-file implementations of deep reinforcement learning algorithms, J. Mach. Learn. Res. 23 (2022) 12585–12602, https://doi.org/10.48550/arXiv.2111.08819.
[70] D. Kingma, J. Ba, Adam: a method for stochastic optimization, Comput. Sci. (2014), https://doi.org/10.48550/arXiv.1412.6980.
[71] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, arXiv preprint arXiv:1707.06347, (2017). 10.48550/arXiv.1707.06347.
[72] A. Bhatt, M. Argus, A. Amiranashvili, T. Brox, Crossnorm: normalization for off-policy td reinforcement learning, arXiv preprint arXiv:1902.05605, (2019). 10.48550/arXiv.1902.05605.
[73] R. Kolisch, Serial and parallel resource-constrained project scheduling methods revisited: theory and computation, Eur. J. Oper. Res. 90 (1996) 320–333, https://doi.org/10.1016/0377-2217(95)00357-6.
[74] S. Hartmann, A competitive genetic algorithm for resource-constrained project scheduling, Naval Res. Logist. 45 (1998) 733–750, https://doi.org/10.1002/(SICI)1520-6750(199810)45:7<733::AID−NAV5>3.0.CO;2-C (NRL).
[75] D. Chen, Y. Lin, W. Li, P. Li, J. Zhou, X. Sun, Measuring and relieving the over-smoothing problem for graph neural networks from the topological view, in: Proceedings of the AAAI Conference on Artificial Intelligence, 2020, pp. 3438–3445.